# Atividade Prática – Agendamento de Tarefas Diárias (App Android)

## Objetivo Geral

Desenvolver um aplicativo Android para agendamento de tarefas diárias, evoluindo gradualmente da manipulação básica de UI até a estruturação com listas e arquitetura MVVM.

## Etapa 1 – Cadastro e Resumo da Tarefa

### Objetivo

Implementar duas telas com passagem de dados via Intent, utilizando diversos componentes visuais.

### Componentes a serem utilizados

- EditText: para digitar o título da tarefa

- Spinner: para selecionar o tipo da tarefa (Ex: Trabalho, Estudo, Lazer, Outro)

- CheckBox ou RadioButton: para indicar se a tarefa é prioritária

- TimePicker: para selecionar o horário da tarefa

- AutoCompleteTextView: para sugestão de local (Ex: Casa, Escritório, Biblioteca, Academia)

- Button: para confirmar/cadastrar a tarefa

- ProgressDialog ou ProgressBar: para simular carregamento ao enviar dados

- Intent: para navegar da Tela 1 para a Tela 2 e passar os dados

### Funcionalidades esperadas

- Tela 1: formulário com os componentes mencionados acima

- Tela 2: exibição do resumo da tarefa cadastrada

- Mostrar abaixo do resumo duas tarefas anteriores como exemplo (pode ser estático, simulado)

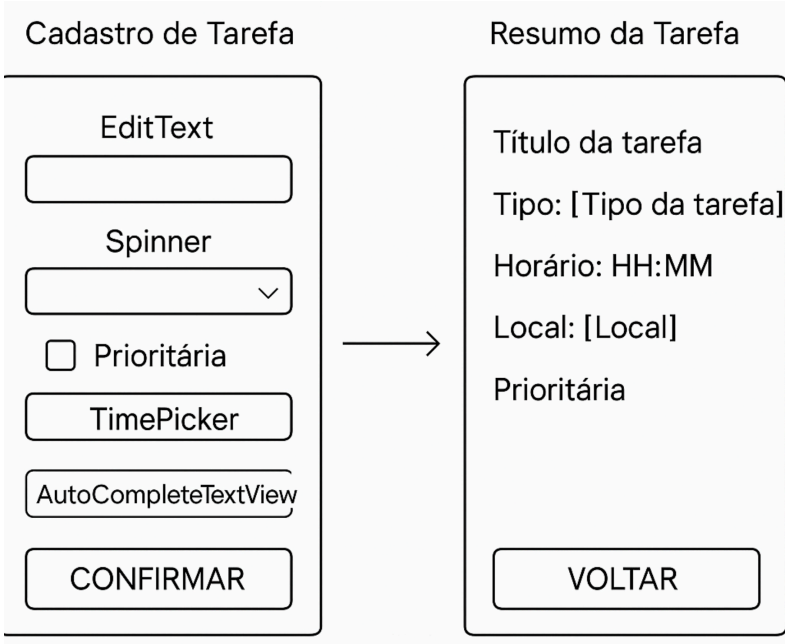
### Observações importantes

- Não é necessário salvar dados entre execuções

- As tarefas anteriores podem ser apenas textos fixos exibidos na segunda tela

- O foco é na manipulação de componentes e intents

**Protótipo**




## Etapa 2 – Lista de Tarefas (ListView ou RecyclerView)

### Objetivo

Permitir visualizar uma lista de tarefas criadas dinamicamente.

### Funcionalidades esperadas

- Adicionar nova tela com ListView ou RecyclerView

- Cada tarefa cadastrada deve ser adicionada a uma lista exibida nessa tela

- Mostrar título, tipo, horário e local da tarefa

- Botão para navegar da Tela 1 ou Tela 2 para a lista completa

### Requisitos Técnicos

- Usar ArrayList<Tarefa> para armazenar as tarefas

- Criar um Adapter personalizado para formatar a exibição de cada item

- Reaproveitar o formulário da Etapa 1

## Etapa 3 – Arquitetura MVVM + LiveData

### Objetivo

Separar responsabilidades da aplicação usando o padrão MVVM e LiveData.

### Funcionalidades esperadas

- Criar ViewModel que mantém a lista de tarefas

- Utilizar LiveData para observar mudanças e atualizar UI

- Organizar pacotes em view, model e viewmodel

- (Opcional) Usar Room para persistência local

### Requisitos Técnicos

- Evitar passagem direta via Intent na Etapa 3 (usar ViewModel)

- Atualizações da lista devem ser automáticas com LiveData

- Projeto deve seguir boas práticas de organização